

CSE471 | WEEK 2 | LECTURE 3

Neural Network Basics

Perceptron, Feedforward Networks & Activation Functions

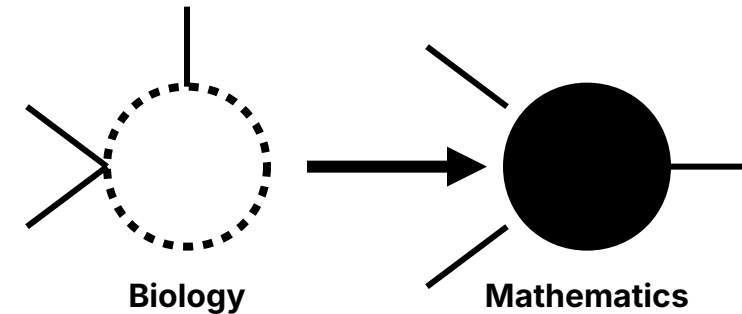
Lovely Professional University

Lecturer : Prof. Rishi Raj

Learning Outcomes

By the end of this lecture, you should be able to:

- └ Explain why **neural networks** were developed to replace manual feature engineering.
- └ Describe the **biological inspiration** behind artificial neurons.
- └ Deconstruct the **Perceptron** model (inputs, weights, and bias).
- └ Design basic **Feedforward Neural Networks**.
- └ Understand and evaluate modern **Activation Functions** (ReLU, Sigmoid, etc.).



Lecture Roadmap

Why Neural
Networks?

Biological
Neurons

Artificial
Neurons

The
Perceptron

Feedforward
Networks

Activation
Functions

Lecture
Summary

How Do We Learn vs. Computers?

How does a child recognize a dog after seeing only a few examples,
while a computer requires thousands of explicit rules?

Human Brain

Neurons Learning Recognition

Computer

Artificial Training Prediction
Nodes

Can machines learn the same way humans do?

Why Neural Networks? (The Shift in AI)

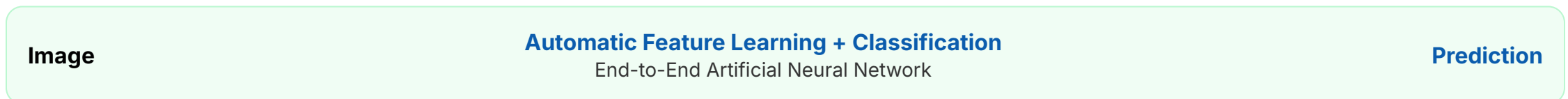
Traditional Machine Learning struggles with complex, unstructured data like images because it relies on **manual feature engineering**.

Traditional Machine Learning (Manual)



Poor scalability, limits performance based on human intuition.

Deep Learning (Neural Networks)



The Biological Inspiration

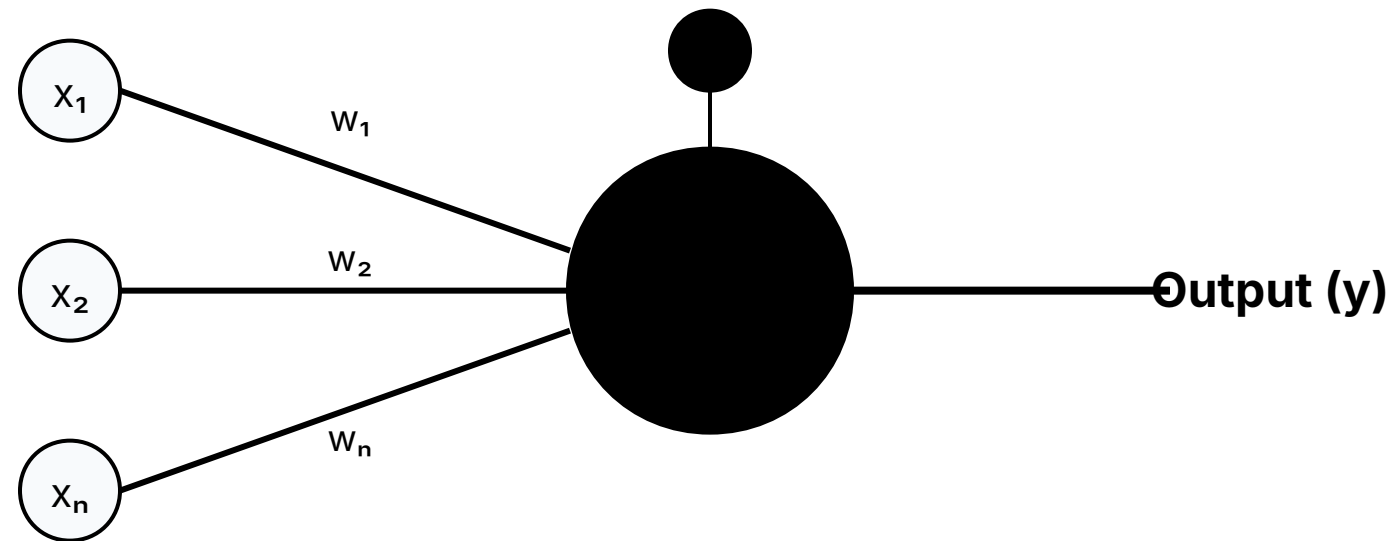
The human brain contains ~86 billion interconnected neurons communicating via electrical and chemical signals.

- └ **Dendrites:** Receives signals from other neurons (**Inputs**).
- └ **Soma (Cell Body):** Processes the incoming signals.
- └ **Axon:** Transmits the signal outward (**Wiring**).
- └ **Synapse:** Connection point varying in strength (**Weights**).

Biological Neuron Diagram

Core Concept: If accumulated signals exceed a threshold, the neuron "fires" an output.

The Artificial Neuron (Mathematical Model)



$$z = \sum_{i=1}^n (w_i \cdot x_i) + b$$

$$y = f(z)$$

The Perceptron

The **Perceptron** is the simplest artificial neural network, essentially a single artificial neuron with a binary step activation function.

- └ Designed for **binary classification** (e.g., 1 or 0).
- └ Learns by adjusting weights iteratively based on output errors.

Did You Know?

Invented in 1958 by Frank Rosenblatt at Cornell Aeronautical Laboratory. It was originally implemented in custom hardware (the Mark I Perceptron) for image recognition, using a 400-pixel visual array connected to motorized potentiometers for weights!

Frank Rosenblatt Mark 1 Perceptron

The Mark I Perceptron Hardware (1958)

Numerical Example: Forward Pass

Let's compute the output of a single neuron manually using a Step Activation (threshold = 0).

1. Given Values

- └ Inputs: $x_1 = 2, x_2 = 3$
- └ Weights: $w_1 = 0.5, w_2 = 0.2$
- └ Bias: $b = 1$

2. Compute Weighted Sum (z)

$$z = (x_1 \cdot w_1) + (x_2 \cdot w_2) + b$$

$$z = (2 \cdot 0.5) + (3 \cdot 0.2) + 1$$

$$z = 1.0 + 0.6 + 1$$

$$\mathbf{z = 2.6}$$

3. Apply Activation

If $z \geq 0$, output 1.
 Else, output 0.

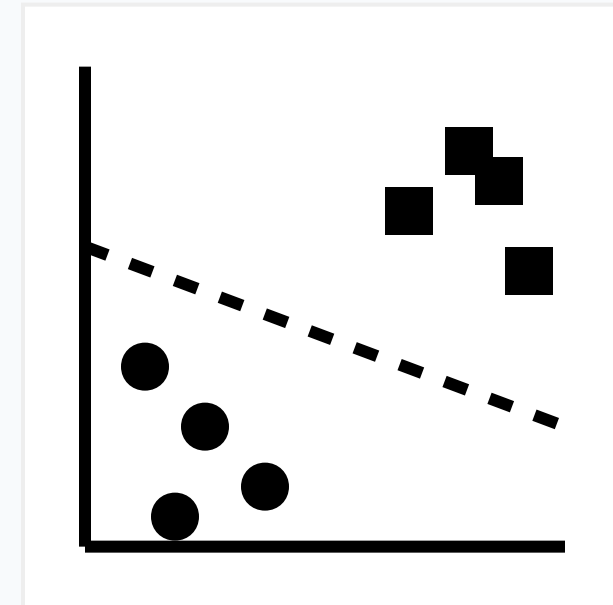
$$2.6 \geq 0$$

Prediction: 1

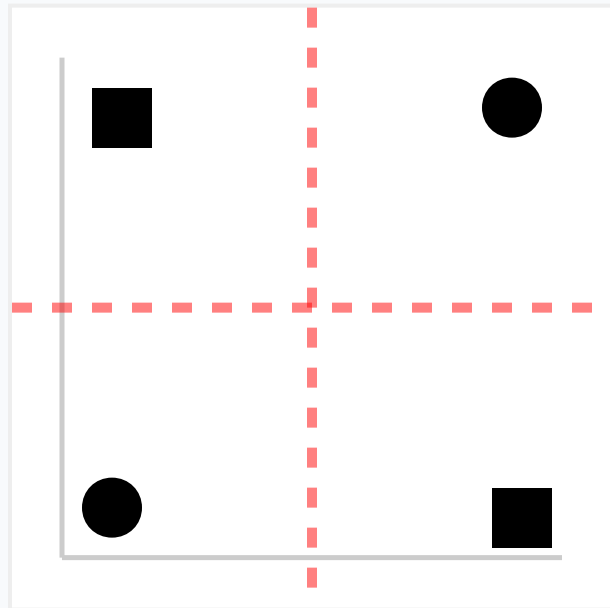
Linear Decision Boundaries

A single neuron mathematically defines a **hyperplane** (a line in 2D space, a flat plane in 3D).

- └ The equation $w_1x_1 + w_2x_2 + b = 0$ is the equation of a straight line.
- └ The perceptron acts as a **Linear Classifier**.
- └ It can only solve problems where data is **Linearly Separable**.



Limitations: The XOR Problem



**Impossible to draw ONE straight line
to separate colors!**

A single perceptron fails spectacularly on simple non-linear logic, such as the **Exclusive OR (XOR)** function.

- └ If data is not linearly separable, a single neuron cannot solve the problem.
- └ **Solution:** We must combine multiple neurons into **Hidden Layers** to draw complex boundaries.

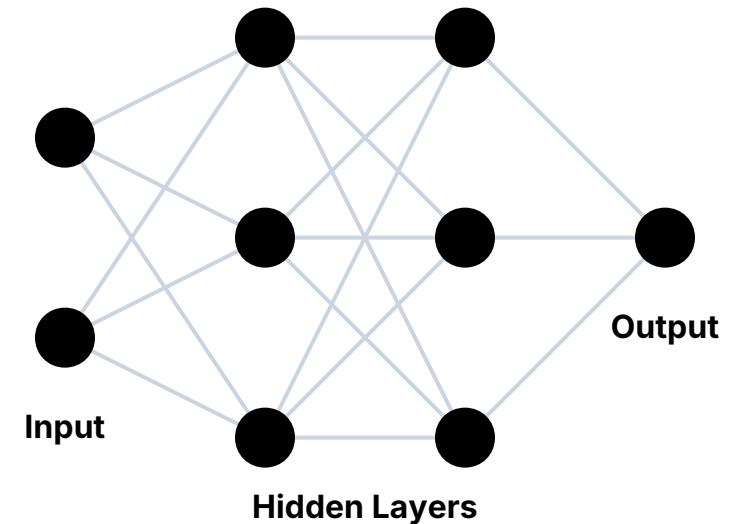
AI Winter

In 1969, Minsky & Papert published a mathematical proof of the perceptron's inability to solve XOR. This caused funding to freeze, triggering the first "AI Winter".

Feedforward Neural Networks

To solve complex non-linear problems, we organize neurons into interconnected layers.

- └ **Input Layer:** Passes raw data into the network (these are not computing neurons).
- └ **Hidden Layers:** Intermediate layers where feature extraction occurs.
- └ **Output Layer:** Produces the final prediction.
- └ **Feedforward:** Information flows in only one direction (Input → Output). No loops.



What makes a network "Deep"?

Shallow Network

1 Hidden Layer. Can solve XOR, but struggles with complex visual data.



Deep Network

Multiple Hidden Layers. Enables **Hierarchical Representation Learning**.



"Deep" simply refers to the number of intermediate hidden layers.

Hierarchical Feature Learning

Deeper networks learn complex concepts by combining simpler ones layer by layer.

Raw Pixels



Layer 1
Edges



Layer 2
Shapes/Corners

Layer 3
Object Parts

Output
Identity (Face)

The Necessity of Activation Functions

Without activation functions, stacking hidden layers is mathematically pointless. A series of linear transformations collapses into a single linear transformation.

$$f_3(f_2(f_1(x))) = W_3(W_2(W_1x)) = (W_3W_2W_1)x = W_{\{net\}}x$$

Without Activation

Network remains entirely linear. Cannot solve XOR or learn complex image features, regardless of depth.

With Activation

Introduces **Non-Linearity**. Allows the network to warp space and draw curved, highly complex decision boundaries.

Popular Activation Functions

Binary Step



Output: {0, 1}. Theoretical use only.

Sigmoid



Output: (0, 1). Vanishing gradients.

Tanh



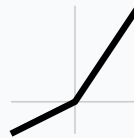
Output: (-1, 1). Zero-centered.

ReLU



Modern default for Hidden Layers.

Leaky ReLU



Solves the "Dying ReLU" problem.

Softmax

$$\frac{e^{z_i}}{\sum e^{z_j}}$$

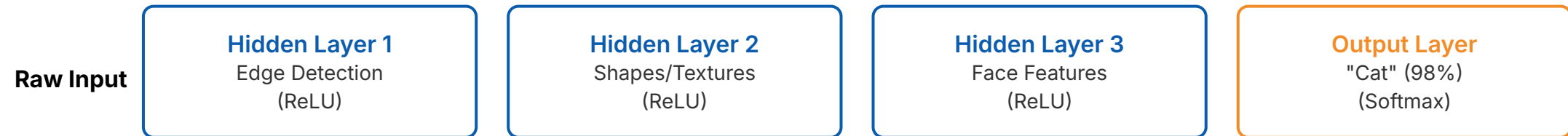
Used in Output Layer for probabilities.

Activation Comparison

Activation	Output Range	Differentiable?	Primary Advantage	Major Limitation	Typical Use
Sigmoid	$(0, 1)$	Yes	Outputs valid probabilities	Vanishing gradients	Binary classification output
Tanh	$(-1, 1)$	Yes	Zero-centered output	Vanishing gradients	Legacy models, RNNs
ReLU	$[0, \infty)$	Yes (mostly)	Fast, avoids vanishing gradient	"Dying ReLU" (outputs stay 0)	Default for Hidden Layers
Leaky ReLU	$(-\infty, \infty)$	Yes	Fixes "Dying ReLU"	Slightly more compute overhead	Hidden layers
Softmax	$(0, 1)$ (sum=1)	Yes	Creates a probability distribution	Computationally heavier	Multi-class output layer

Putting it Together: Cat Classifier

How an end-to-end Feedforward architecture learns representations.



Lecture Summary

1. Biological Inspiration

Neural networks are mathematical abstractions of brain processes.

2. The Core Engine

Neurons compute a weighted sum. Weights = Importance, Bias = Threshold shift.

3. Depth Matters

Hidden layers allow for hierarchical feature learning, solving non-linearities like XOR.

4. Activation is Key

Introduces essential non-linearity. ReLU is the standard for modern deep networks.

Interactive Task & References

Think-Pair-Share (3 mins)

Why exactly can't a single perceptron solve the XOR problem?

Homework

Design a perceptron with 3 inputs, custom weights, and a bias. Compute outputs manually for three different input combinations.

References

- └ **Original Paper:** Frank Rosenblatt, *The Perceptron* (1958).
- └ **Textbook:** Ian Goodfellow, *Deep Learning* (Chapter 6).
- └ **Textbook:** Michael Nielsen, *Neural Networks and Deep Learning*.
- └ **Course Inspiration:** Stanford CS231n Lecture Notes.
- └ **Frameworks:** PyTorch & TensorFlow Documentation.